

F#

История и особенности · Запуск программы
· Типы и переменные · Функции, рекурсия
и замыкания

**Краткий обзор языка программирования F# по
структуре примера**

Мини-пример

```
let names = [ "Анна"; "Игорь" ]  
  
names  
|> List.map (fun n -> $"Привет, {n}")  
|> List.iter (printfn "%s")
```

История

- Разработка F# началась в Microsoft Research Cambridge под руководством Дона Сайма.
- Язык вырос из ML/OCaml-традиции, но был спроектирован для платформы .NET.
- Ключевые черты F# формировались в 2004-2007 годах; выпуск F# 2.0 связан с продуктовой поддержкой в Visual Studio.
- Сегодня F# - открытый, кроссплатформенный язык экосистемы .NET.



Особенности и спецификация

λ

Функциональность

Функции первого класса, композиция, рекурсия, замыкания.

T

Статическая типизация

Типы обычно выводятся компилятором, но проверяются до запуска.

□

Иммутабельность

let-значения неизменяемы по умолчанию; mutable используется явно.

{ }

Богатые типы

Записи, кортежи, размеченные объединения, option/result.

↔

Совместимость с .NET

Можно вызывать C# библиотеки и использовать CLR, NuGet и .NET SDK.

10

Спецификация

Актуальная спецификация развивается через документ и RFC; F# 10 поставляется с .NET 10.

Синтаксис в одну строку

```
let square x = x * x
let squares = [1..5] |> List.map square
printfn "%A" squares // [1; 4; 9; 16; 25]
```

Способы запуска - сравнение

Три варианта для демонстрации: локальный SDK, Docker и онлайн-среда. Выбор зависит от установки, воспроизводимости и ограничений среды.

Способ	Плюсы	Минусы
Локальный .NET SDK	Полный контроль, F# Interactive, IDE, NuGet, проекты .fsproj	Нужно установить SDK и редактор
Docker	Одинаковая версия SDK у всех участников, чистая среда	Нужен Docker, медленнее старт контейнера
Онлайн-среда	Быстро для защиты, не нужна установка	Ограничения ввода-вывода, пакетов и версий

Запуск через Docker

- Для F# используются официальные образы .NET SDK от Microsoft.
- Подходит для воспроизводимой версии компилятора и FSharp.Core.
- Можно запускать как интерактивный REPL, скрипт .fsx или проект .fsproj.

Быстрый запуск F# Interactive

```
docker run -it --rm \
  mcr.microsoft.com/dotnet/sdk:10.0 \
  dotnet fsi
```

Dockerfile проекта

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /app
COPY . .
RUN dotnet publish -c Release -o out

FROM mcr.microsoft.com/dotnet/runtime:10.0
WORKDIR /app
COPY --from=build /app/out .
ENTRYPOINT ["dotnet", "App.dll"]
```

Онлайн-среда и локальное окружение

Онлайн

- Fable REPL - запуск F# в браузере через трансляцию в JavaScript.
- Binder/Jupyter - удобно для учебных ноутбуков и демонстраций.
- Онлайн-среды быстрее стартуют, но ограничены по пакетам и файловой системе.

Локально

- Установить .NET SDK: `dotnet --version`.
- Запуск скрипта: `dotnet fsi script.fsx`.
- Создать проект: `dotnet new console -lang F#`.
- Редакторы: VS Code + Ionide, Visual Studio, Rider.

Минимальный проект

```
dotnet new console -lang F# -o Demo
cd Demo
dotnet run
```

Типы данных

F# статически типизирован: типы обычно выводятся автоматически, но ошибки типов обнаруживаются до запуска.

int / int64

целые числа

float / decimalвещественные и
финансовые значения**bool**

true / false

string / char

строки и символы

unit

нет полезного результата

tuple

несколько значений вместе

record

именованные поля

DU / option

варианты данных

list / array / seq

коллекции

Map / Set

словарь и множество

Пример типов

```
let age = 20           // int
let price = 19.99M     // decimal
let name = "F#"        // string
let point = (10, 20)   // tuple
type User = { Name:string; Age:int }
type Status = Active | Blocked
```

Операции над типами



Арифметика

Операторы работают с совместимыми числовыми типами.



Строки

Интерполяция строк и функции модуля String.



Сравнение

Структурное сравнение для многих встроенных типов.



Конвейер

Передаёт результат слева как аргумент функции справа.



Композиция

Собирает несколько функций в одну.



Коллекции

map, filter, fold часто заменяют ручные циклы.

Пример

```
let total = [1; 2; 3; 4]
  |> List.filter (fun x -> x % 2 = 0)
  |> List.map (fun x -> x * x)
  |> List.sum

let message = $"sum = {total}"
```

Приведение типов

- F# не полагается на широкие неявные преобразования: совместимость типов важна.
- Обычные преобразования пишутся явно: int, float, decimal, string.
- Тип можно уточнить аннотацией: let x: int = 42.
- Для иерархий .NET доступны upcast/downcast и операторы :> / :?>.

Примеры

```
let a = int "42" + 1      // 43
let b = float 10 / 4.0    // 2.5
let c: decimal = decimal 19.99
let s = string c

// "10" + 5 // ошибка типов
```

Приведение в .NET-иерархии

```
let o: obj = box 123
let n = unbox<int> o

let stream: System.IO.Stream =
    new System.IO.MemoryStream() :>
    System.IO.Stream
```

Области видимости

- let создает имя в текущей лексической области видимости.
- Отступы и вложенные выражения задают структуру блока.
- Имя можно затенить: новое значение скрывает старое внутри области.
- mutable нужен только для изменяемой переменной; по умолчанию значения неизменяемы.

Пример

```
let x = 10

let result =
  let x = 20    // затенение
  x + 1

printfn "%d" x    // 10
printfn "%d" result // 21

let mutable counter = 0
counter <- counter + 1
```

Владение и передача владения

- В F# нет модели владения как в Rust: управление памятью выполняет CLR и сборщик мусора.
- Значимые типы обычно копируются, ссылочные типы передаются как ссылки на объект.
- Иммутабельность снижает риск случайно изменить данные через общую ссылку.
- Для ресурсов вне памяти используется IDisposable и конструкция use.

Не изменение, а новая запись

```
type User = { Name:string; Age:int }  
  
let u1 = { Name = "Ada"; Age = 20 }  
let u2 = { u1 with Age = 21 }  
  
printfn "%A" u1 // Age = 20  
printfn "%A" u2 // Age = 21
```

Ресурсы

```
let readFirstLine path =  
    use reader = System.IO.File.OpenText(path)  
    reader.ReadLine()
```

Функции: объявление и вызов

- Функции объявляются через `let` и часто не требуют явных типов.
- Функции - значения первого класса: их можно хранить, передавать и возвращать.
- Анонимная функция пишется через `fun`.
- Конвейер `|>` делает цепочки вызовов читаемыми.

Пример

```
let greet name = $"Hello, {name}"
```

```
let twice f x = f (f x)
```

```
let inc x = x + 1
```

```
let result = twice inc 10 // 12
```

```
["F#"; ".NET"]
```

```
|> List.map greet
```

```
|> List.iter (printfn "%s")
```

Входные данные функций

- Обычная форма - каррированные параметры:
let add x y = x + y.
- Типы можно выводить автоматически или задавать явно.
- Кортежный параметр полезен, когда логически передается одна пара/структура.
- Для методов .NET доступны именованные и optional-параметры.

Каррирование и типы

```
let add x y = x + y
let add10 = add 10

printfn "%d" (add10 5) // 15

let distance (x1:float, y1:float) (x2, y2) =
    sqrt ((x2-x1)**2.0 + (y2-y1)**2.0)
```

Optional-параметр метода

```
type Printer() =
    member _.Print(text, ?prefix) =
        let p = defaultArg prefix ""
        printfn "%s%s" p text

Printer().Print("done", prefix = "> ")
```

Выходные данные функций

- Функция возвращает значение последнего выражения - return обычно не нужен.
- Несколько результатов удобно вернуть кортежем.
- Отсутствие результата часто моделируется через option.
- Ошибочный/успешный результат можно выразить через Result.

Пример

```
let minmax numbers =
  (List.min numbers, List.max numbers)

let tryDivide a b =
  if b = 0 then None
  else Some (a / b)

let lo, hi = minmax [3; 1; 4; 1; 5]
printfn "%d %d" lo hi    // 1 5
```

Рекурсия

- Рекурсивная функция объявляется через `let rec`.
- Обычная рекурсия оставляет работу после возврата вызова.
- Хвостовая рекурсия переносит накопление в параметр-аккумулятор.
- В F# 10 улучшена поддержка хвостовых вызовов в `computation expressions`.

Обычная рекурсия

```
let rec fact n =
    if n = 0 then 1
    else n * fact (n - 1)

printfn "%d" (fact 5) // 120
```

Хвостовая рекурсия

```
let factTail n =
    let rec loop n acc =
        if n = 0 then acc
        else loop (n - 1) (n * acc)
    loop n 1

printfn "%d" (factTail 5) // 120
```

Замыкания

- Замыкание - функция, которая использует значения из внешней области видимости.
- Каждый вызов внешней функции может создать независимое состояние.
- Замыкания применяются для конфигурации функций, инкапсуляции состояния и callback-логики.
- В F# это естественно, потому что функции являются значениями первого класса.

Пример

```
let counter start =
    let mutable n = start
    fun () ->
        n <- n + 1
        n

let c1 = counter 0
let c2 = counter 100

printfn "%d %d %d" (c1()) (c1()) (c2())
// 1 2 101
```

Спасибо за внимание

Вопросы?

Итоговый пример F#

```
type Result = Ok of string | Error of string

let handle value =
    match value with
    | Ok text -> printfn "Готово: %s" text
    | Error msg -> printfn "Ошибка: %s" msg
```